Valeo

THESIS PROPOSAL

Farida Elgharabawy - Hana Shalaby - Haya Shalaby -
Hoda Hussein - Laila Sayed - Mohamed El Refai

OVERVIEW

1. Motivation
2. Problem & Solution
3. Literature Review
5. Methodology
6. Initial Design
7. Timeline
8. Outcomes

MOTIVATION

- LLM performance does not only mean model size
- Context quality is critical
- This is the Context Management Problem
- Enterprise AI assistants are now standard (Valeo)
- In enterprise settings, hallucinations and reasoning degradation create a high risk

As a result…

- Our goal is to build a context engineering optimization Agent
- Acts as a middleware layer between Valeo’s codebase and internal AI agents

THE PROBLEM

Context Engineering for AI agents in Software Development of Large Codebases

AI agents in enterprises frequently generate code that looks correct, but is wrong in enterprise settings.

Why?

Current agents do not have a structured method to decide what context is important and what is noise in large codebases.

Consequently...

Developers waste time debugging, rejecting suggestions, and manually correcting hallucinations.

This defeats the purpose of making the process easier for them.

THE SOLUTION

A layer that sits between the user and the LLM

- The goal is to make AI agents produce code that does not only have a correct syntax, but also is valid relative to the context types available in enterprises such as Valeo (API contracts, Business Logic, ...etc).
- The layer will retrieve, prune, and structure the right context from the prompt before it is injected into the AI agent used by the enterprise.

LITERATURE REVIEW

Context Pruning Methods

Study	Problem Addressed	Core Approach	Key Results
Li et al., 2024 *Selective Context*	Long prompts contain redundant tokens that waste context space and compute.	Sscore every token, groups them into phrases, and drops the least surprising ones.	Marginal quality loss until very aggressive filtering; optimal cutoff score is task-dependent.
Jiang et al., 2023; Pan et al., 2024 *LLMLingua / LLMLingua-2*	Token-level filtering lacks faithfulness guarantees; compressed prompts may distort meaning.	LLMLingua-2 distils GPT-4 into a dedicated compression model optimising compactness, informativeness, and faithfulness.	Up to 20× compression; no tuning needed
Anagnostidis et al., 2023 *Dynamic Context Pruning*	Even after a short prompt, the model keeps looking back at old irrelevant tokens on every single generation step.	Prune unnecessary past tokens at each transformer layer during decoding.	80% of context pruned with negligible loss

Context Injection Strategies

Study / Source	Problem Addressed	Core Approach	Key Results
Liu et al., 2024 *Lost in the Middle*	The model pays strong attention to the start and end of a prompt, but largely ignores what's in the middle.	Ran experiments placing the answer at different positions in a long prompt and measured how often the model found it.	Place critical info at the start or end of the prompt.
Rajasekaran et al., 2025 *Anthropic Context Guide*	When instructions and data are mixed together, the model gets confused about what's a command and what's just content.	Clearly label each section with XML tags: this is the system role, this is the task, this is the data. Load documents only when actually needed, not all upfront.	Less confusion, less injection risk, better focus. No industry-wide standard format yet.
Microsoft Research, 2025 *Modular Assembly*	In multi-step AI agents, context becomes a messy pile of instructions, reasoning notes, and tool outputs all thrown together.	Each piece of context gets its own clearly labelled block, added only at the step where the agent actually needs it.	Cleaner reasoning across long tasks, but adding more labelled blocks can make the prompt long again, partially undoing pruning.

LITERATURE REVIEW

Hybrid Retrieval Strategies

Study	Problem Addressed	Core Approach	Key Results
Li et al. *SELF-ROUTE*	LC beats RAG but costs far more; 63% of queries return identical results from both.	Try RAG first; escalate to full LC only if the LLM calls the query unanswerable. No fine-tuning.	Near-LC accuracy while routing the majority to cheaper RAG, with negligible overall quality drop.
Jiang et al. *LongRAG*	100-token passages cause low recall and expose the reader to confusing irrelevant chunks.	Group related docs into 4–6 K-token units,and feeding units to a long-context reader.	Matches supervised state-of-the-art without fine-tuning.
Lu et al. *TurboRAG*	RAG recomputes KV caches from scratch every query, making TTFT unacceptably high.	Pre-compute document KV caches offline; concatenate at inference	TTFT reduced up to 9.4×; accuracy within 1% of standard RAG.
Ge et al. *FastGen*	KV cache memory grows rapidly with model size, forcing costly offloading.	Profile each attention head during prompt encoding to identify its structural pattern, then apply the most memory-efficient compression policy per head. Profile is computed once and remains valid throughout decoding.	30–40% memory reduction with negligible quality loss; beats all fixed compression baselines.

LITERATURE REVIEW

Industry Products

Product	Problem	Approach	Results / Limits
LangChain, 2026 *LangChain*	Finite context windows and static knowledge limit LLM usefulness on live enterprise data.	Modular RAG pipeline: load → chunk → embed → retrieve. Supports both standard two-step RAG and agentic RAG, where the model dynamically decides when and what to retrieve.	Flexible, widely used. Good for most RAG and hybrid setups.
LlamaIndex, 2026 *LlamaIndex*	Keyword search misses meaning; semantic search misses exact terms.	Indexes documents once by meaning, and once by keywords. Then it combines the results from both searches using a scoring formula.	Best-in-class hybrid retrieval; domain-optimisable
Shah, 2026 *Sourcegraph Cody*	Enterprise codebases span many repos, which is too large for any single context window.	RAG across the entire codebase. It also connects to tools like Jira and Notion so it has project context too, not just code.	Excellent code comprehension. Limits: weak auto-review; truncates on large diffs; uneven IDE support.

METHODOLOGY

The Context Optimization Engine

Technical Pipeline

Hybrid Retrieval: Merging RAG and Long Context to maximize recall while controlling token overhead.

Systematic Pruning: Removing up to 80% of "noise" using Selective Context based on token self-information.

Structured Injection: Using XML tagging to mitigate "Lost-in-the-Middle" positional biases in long prompts.

Operational Approach

Hard Prompting: Focusing on input-level optimization since proprietary models like GPT/Gemini prevent weight access.

Enterprise Grounding: Prioritizing internal API contracts and business logic over general public knowledge.

SYSTEM ARCHITECTURE

Agentic Workflow & Performance Optimization

Agentic Framework

Supervisor Architecture: A master agent decomposes complex coding tasks into modular, specialized sub-tasks.

Refining Loop: Implementing a cycle of Planning, Information Retrieval, and Self-Reflection to reduce hallucinations.

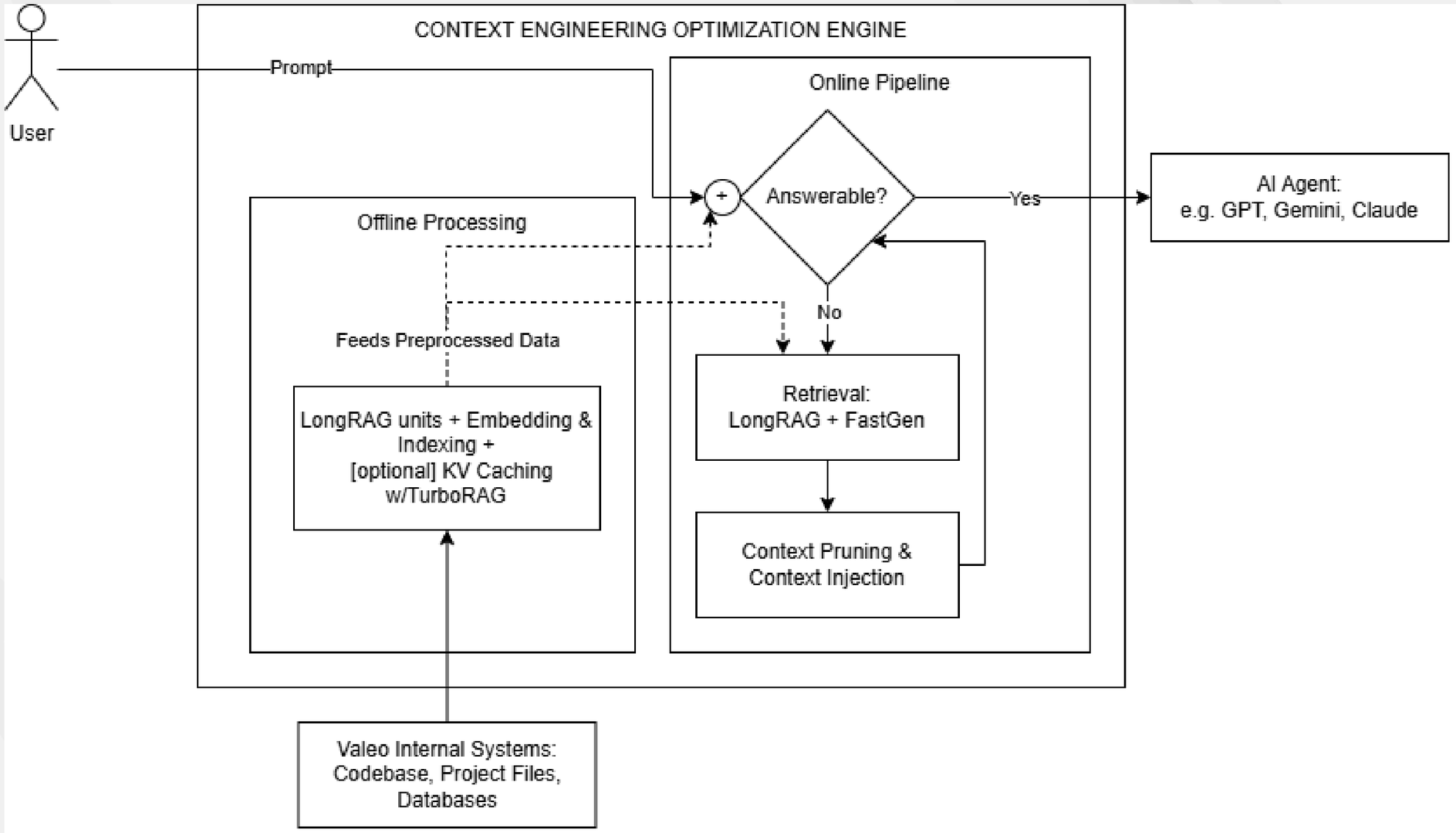
Inference Efficiency

KV Cache Optimization: Applying adaptive compression to reduce GPU memory pressure and improve TTFT.

Unified Evaluation: Benchmarking across GPT-4o and Gemini 1.5 using an LLM-as-a-judge protocol calibrated for precision.

SYSTEM ARCHITECTURE

Block Diagram



TIMELINE

Mid March
Module 1: A research report on "RAG Techniques vs. Long Context" performance for sofware development in large codebases

Mid April
Module 2A: Developing a Context Pruning algorithm

First Week of May
Module 2B: Implement Stuctured Context Injection. Submit a prototype middleware (Python/TypeScript) that intercepts the user prompt and optimizes the input context before sending it to the LLM.

First Week of October
Module 3A: Create a "Golden Dataset" of 50 typical Valeo dev tasks

End of November
Module 3: Run A/B tests and measure metrics. Submit final "Accuracy Optimization Strategy" guide and the benchmark results.

CAPSTONE EMPOWERMENT OUTCOMES

Commerical Product



Research

COMMERICAL PRODUCT

1- Our product is middleware prototype it's a functional software artifact with an integration point
(IDE → Engine → LLM)

2-The problem we are solving is a recognized industry pain point with clear commercial value, as companies already pay for tools like GitHub Copilot and Cursor.

3-It is being built for and validated against an actual enterprise client (Valeo), meaning it has a real user, a real use case, and a real deployment environment.

RESEARCH REPORT

1- The comparison between RAG and Long-Context Caching

2-Controlled golden dataset of 50 representative tasks, running a raw input vs. optimized context

3-Measuring quantifiable metrics like hallucination rate, compilability, and style guide adherence

THANK YOU

September 2030